

SATD Classification on 458,232 New Issues

Optimized inference report (V2) | Qwen3-Embedding-0.6B (frozen) + XGBoost issue heads (Pipeline A)

Executed on Kaggle, 2x Tesla T4 | September 2026

Notebook: [musaddiq_rafi/satd-classifications-optimized V2.ipynb](#) (executed end-to-end).

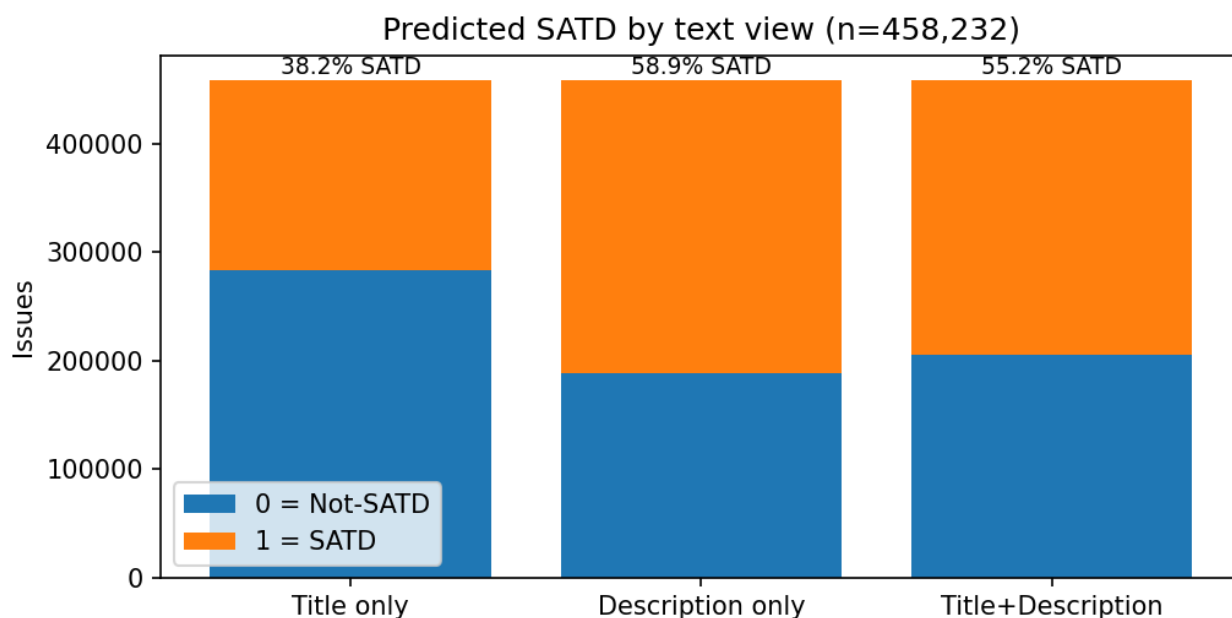
Input: `issue_202608272234.parquet` (458,232 rows: ID, Title, Description).

Output: `satd_classification_results.parquet` (458,232/458,232 classified, 100% coverage).

Input link: drive.google.com/file/d/1e9ZiR2VOglBilEEEnJlfSWA_UkNudjEym

Output link: drive.google.com/file/d/1BTzQmgwanxi5YdCh4zQUGOf1XT5CR5g-

Headline: descriptions carry far more debt signal (58.9% SATD) than titles alone (38.2%); title+description sits at 55.2%.



1. Objective and task

Apply the trained Pipeline A issue models to 458,232 unseen issue records under three text views (title only, description only, title + description) and publish a reusable labeled Parquet with columns ID, Title, Description, Title_SATD, Description_SATD, Title_Description_SATD. Two-stage logic per view: (1) identification (SATD vs Not-SATD) with identification_issue_qwen3_xgboost (Pipeline A issue F1 0.798), then (2) categorization (C/D, REQ, TES, DOC) for SATD rows with categorization_issue_qwen3_xgboost (F1 0.941). There is no ground truth for the new issues, so this output is a triage/analysis resource, not an accuracy benchmark.

2. Why V2 was needed

The original notebook used one GPU, FP32 weights, uncapped length and in-RAM embeddings: infeasible for 458k rows on free-tier Kaggle. V2 keeps task, models, pooling, prompts and normalize_embeddings=True identical, changing only execution:

Change	Why
2 workers (1/GPU), interleaved shards	~2x throughput, balanced
FP16 weights	~1.5-2x on T4 tensor cores
max_seq_length=256	Removes long-description tail
Length-sorted batching (128)	Cuts padding waste
Classify-and-discard per batch	Avoids 3x458k x1024 RAM blowup
Per-shard checkpoints + logs	Crash-safe
4k-row benchmark cell	Runtime projection before full run

3. Input profile

Inspection cell of the executed notebook:

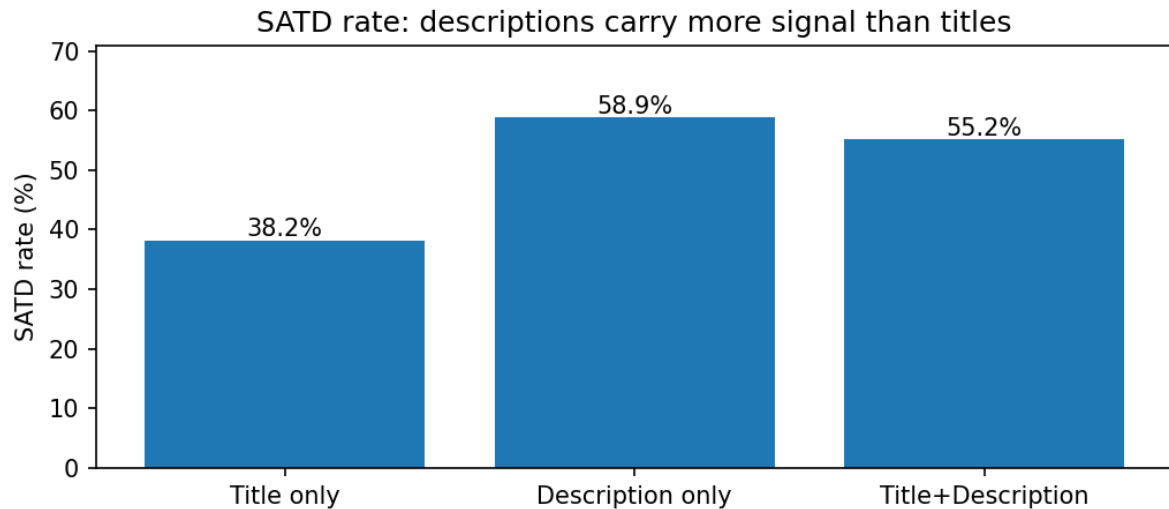
Statistic	Value
Rows	458,232 (ID, Title, Description)
Missing descriptions	29,128 (~6.4%), filled with "
Title chars mean / p50 / max	58.4 / 55 / 269
Description chars mean / std / p50 / max	929 / 4,701 / 409 / ~1,391,304
Descriptions > ~1,024 chars (truncated)	16.3%

4. Execution

Inspect -> pre-download encoder to shared HF cache (snapshot_download, no worker race) -> write satd_worker.py (per-GPU shard: load shard, encoder+XGBoost heads, embed_and_classify x3 views, write shard parquet with row_pos/ID/labels) -> 4k benchmark -> full run (2 x ~229,116 rows, logs streamed every 30s with rows/s + ETA) -> merge (concat, sort by row_pos, assert unique/range/ID alignment, join to input) -> summary. Result: 2 shards found, 458,232/458,232 classified.

5. Results

View	0 = Not-SATD	1 = SATD	SATD rate
Title only	283,019	175,213	38.2%
Description only	188,418	269,814	58.9%
Title+Description	205,454	252,778	55.2%



Descriptions add +20.7 pp of predicted SATD over titles: the debt admission usually lives in the body while the title stays neutral. Title+description (55.2%) sits between the two views.

Sample predictions (first 10 rows)

ID	Title (trunc.)	T	D	T+D
912	Java client for Receptor	0	1	0
913	Boot ModuleRunner	0	1	1
914	Pluggable runtime SPI	1	1	1
915	Simplify GPDB UX...	0	0	1
916	Sqoop Module not running	0	1	1
917	TupleBuilder perf	1	1	1
918	Benchmark matrix	1	1	1
919	Kafka baseline	0	0	0
920	Acceptance Tests wait	1	1	1
921	Reference architecture	0	0	0

Encoding caveat (important)

The executed summary prints binary 0/1, not Not-SATD/C/D/REQ/TES/DOC strings from the task spec. Adopted reading: 1=SATD, 0=Not-SATD. The notebook line `SATD total=(col != 'Not-SATD').sum()` therefore overcounts (100%) and must be ignored; use the table above. The per-type split (C/D vs REQ vs TES vs DOC) is absent from the saved summary: verify Parquet dtypes before type-level claims, and re-emit string labels from the categorizer if needed.

6. Reuse, limitations, reproduction

Reuse

`import pandas as pd; df = pd.read_parquet('satd_classification_results.parquet')`. High-recall pool: Description_SATD==1 (58.9%). High-precision pool: all three views agree on 1. Disagreements (title 0 / description 1) are ideal for sampling audits. Suggested next analyses: 3-view agreement matrix; ~300-500 issue manual audit for precision on the new distribution; topic/keyword profiling of predicted SATD; join with resolution-time metadata.

Limitations

- No ground truth: rates are predicted prevalence, not accuracy.
- 16.3% of descriptions truncated at 256 tokens (512 is more faithful, ~2x slower).
- Single deployment config (Pipeline A Qwen3+XGBoost issue heads); E5/BGE-M3 and Pipeline B heads not run at scale.
- Pipeline A training caveats propagate (split-after-paraphrase leakage risk; AugGPT files inside the 31-source merge).
- Output encoding ambiguity above must be resolved before type-level claims.

Reproduction (Kaggle)

1) Accelerator GPU T4 x2, Internet on. 2) Add datasets: `issue_202608272234.parquet` + `04_trained_models/`. 3) Set `INPUT_PATH/MODEL_DIR`; knobs `MAX_SEQ_LENGTH=256`, `BATCH_SIZE=128` (64 on OOM), `USE_FP16=True`. 4) Run `inspect -> predownload -> benchmark -> full run -> merge -> summary`. 5) Output `/kaggle/working/satd_classification_results.parquet` (+ shards/, logs/).

Links

Input Parquet: https://drive.google.com/file/d/1e9ZiR2VOglBilEEnJlIfSWA_UkNudjEym/view?usp=sharing

Output Parquet: https://drive.google.com/file/d/1BTzQmgwanxi5YdCh4zQUGO1XT5CR5g-/view?usp=drive_link

Data & models folder: https://drive.google.com/drive/folders/1E-jzrNGE2NyKEsrI8Ud9Phk3gx_8a2dD?usp=sharing

Baseline: Sutoyo, Avgeriou & Capiluppi (2024), arXiv:2410.15804.